Sleep-EDF: финальное решение

Классификация стадий сна по EEG и EOG. Основное решение157 объединяет U-Sleep и ансамбль113 с равными весами. Веса моделей зафиксированы; обучение при запуске не выполняется.

Результаты: `submission.csv` —157; при `OUTPUT_SECOND_SUBMISSION=True` дополнительно создаётся `submission_2.csv` —113.

Среда: Kaggle T4×2, Internet Off. Подключите исходный Sleep-EDF Dataset version1, `sleep-edf-final-157-113-inputs.zip` и прежний `sleep-edf-final-runtime-linux.zip`. Выполните все ячейки последовательно. В конце сохраняются CSV, отчёт о времени и проверке воспроизводимости, ZIP результатов.

In [1]:

```
import importlib.util
import json
import os
import subprocess
import sys
import time
from pathlib import Path

NOTEBOOK_STARTED = time.perf_counter()
OUTPUT_SECOND_SUBMISSION = False
DEVICE = "cuda:0"
THREADS = max(1, os.cpu_count() or 2)
FEATURE_WORKERS = min(2, THREADS)
OUTPUT_ROOT = Path("/kaggle/working/sleep-edf-final-157-113")
FINAL_ROOT = None  # Автопоиск нового Input bundle; GitHub и токен не нужны.
DATA_ROOT = None  # Автопоиск исходного full_psg_v1 version1.
print(
    "Дополнительный CSV113:", OUTPUT_SECOND_SUBMISSION, "· GPU:", DEVICE, "· CPU threads:", THREADS
)
RUNTIME_INPUT = None  # Автопоиск отдельного offline-runtime Input.
RUNTIME_CACHE = Path("/kaggle/working/_sleep-edf-offline-runtimes")
```

```
Дополнительный CSV113: False · GPU: cuda:0 · CPU threads: 4
```

In [2]:

```
import hashlib
from zipfile import ZipFile


def unpack(path):
    key = hashlib.sha256(
        (str(path) + str(path.stat().st_size) + str(path.stat().st_mtime_ns)).encode()
    ).hexdigest()[:16]
    target = Path("/kaggle/working/_final_input") / key
    if (target / ".unpacked").exists():
        return target
    target.mkdir(parents=True, exist_ok=True)
    with ZipFile(path) as z:
        for info in z.infolist():
            dest = (target / info.filename).resolve()
            if (
                not dest.is_relative_to(target.resolve())
                or (info.external_attr >> 16) & 0o170000 == 0o120000
            ):
                raise ValueError("Unsafe ZIP member")
            z.extract(info, target)
    (target / ".unpacked").touch()
    return target


if FINAL_ROOT is None:
    candidates = [
        p.parent
        for p in Path("/kaggle/input").rglob("bundle.json")
        if json.loads(p.read_text()).get("kind") == "sleep_edf_final_v3"
    ]
    if not candidates:
        for archive in Path("/kaggle/input").rglob("*.zip"):
```

```
            with ZipFile(archive) as z:
                if (
                    "bundle.json" in z.namelist()
                    and json.loads(z.read("bundle.json")).get("kind")
                    == "sleep_edf_final_v3"
                ):
                    candidates.append(unpack(archive))
    if len(candidates) != 1:
        raise RuntimeError(
            "Нужен новый sleep-edf-final-157-113-inputs bundle; прежний Input v1 не подходит"
        )
    FINAL_ROOT = candidates[0] / "final"
BUNDLE_ROOT = FINAL_ROOT.parent
manifest = json.loads((BUNDLE_ROOT / "bundle.json").read_text())
for name, expected in manifest["files"].items():
    path = (BUNDLE_ROOT / name).resolve()
    if not path.is_relative_to(BUNDLE_ROOT.resolve()):
        raise ValueError("Unsafe bundle path")
    with path.open("rb") as f:
        actual = hashlib.file_digest(f, "sha256").hexdigest()
    if actual != expected:
        raise ValueError(f"Input checksum mismatch: {name}")
if DATA_ROOT is None:
    roots = [
        p.parent
        for p in Path("/kaggle/input").rglob("MANIFEST.json")
        if (p.parent / "epochs/test").is_dir()
        and (p.parent / "sample_submission.csv").is_file()
    ]
    if len(roots) != 1:
        raise RuntimeError(f"Нужен один исходный full_psg_v1: {roots}")
    DATA_ROOT = roots[0]
print("Код/веса:", FINAL_ROOT, "\nДанные:", DATA_ROOT)
print("Исходный dataset: levshaazz/sleep-edf-full-psg-course-data, version1")
```

```
Код/веса: /kaggle/input/datasets/sevaperetiatko/sleep-edf-final-offline-inputs/final
Данные: /kaggle/input/datasets/levshaazz/sleep-edf-full-psg-course-data
Исходный dataset: levshaazz/sleep-edf-full-psg-course-data, version1
```

In [3]:

```
import html

from IPython.display import HTML, display

spec = importlib.util.spec_from_file_location(
    "final_notebook_process", FINAL_ROOT / "notebook_process.py"
)
process_module = importlib.util.module_from_spec(spec)
spec.loader.exec_module(process_module)
run_command = process_module.run_command
process_module.reset_commands()

runtime_spec = importlib.util.spec_from_file_location(
    "sleep_edf_offline_runtime", FINAL_ROOT / "offline_runtime.py"
)
runtime_module = importlib.util.module_from_spec(runtime_spec)
runtime_spec.loader.exec_module(runtime_module)
if RUNTIME_INPUT is None:
    RUNTIME_INPUT = runtime_module.discover(Path("/kaggle/input"))
print("Offline runtime Input:", RUNTIME_INPUT)
handle = display(HTML("<b>Подготовка offline-окружений</b>"), display_id=True)


def show_runtime_stage(text):
    handle.update(HTML("<b>Подготовка offline-окружений</b><br>" + html.escape(text)))


runtimes, runtime_manifest = runtime_module.prepare(
    RUNTIME_INPUT, RUNTIME_CACHE, FINAL_ROOT, show_runtime_stage
)
FEATURE_PYTHON = runtimes["features"]
BASE = [str(runtimes["inference"]), "-I"]
ENV = {
    "PYTHONHOME": "",
    "PYTHONPATH": "",
    "VIRTUAL_ENV": "",
    "PYTHONNOUSERSITE": "1",
    "UV_OFFLINE": "1",
    "UV_PYTHON_DOWNLOADS": "never",
    "PIP_NO_INDEX": "1",
    "CUBLAS_WORKSPACE_CONFIG": ":4096:8",
```

```
    "MPLBACKEND": "Agg",
    **{
        k: str(THREADS)
        for k in ("OMP_NUM_THREADS", "MKL_NUM_THREADS", "OPENBLAS_NUM_THREADS")
    },
}
OUTPUT_ROOT.mkdir(parents=True, exist_ok=True)
(OUTPUT_ROOT / "offline_runtime_manifest.json").write_text(
    json.dumps(runtime_manifest, indent=2) + "\n"
)
subprocess.run(["nvidia-smi"], check=True)
print("Notebook kernel:", sys.version)
print(
    "Восстановлены Python3.12.12 и3.14.2 с пакетами из Input. Скачиваний и pip/uv install нет."
)
```

```
Offline runtime Input: /kaggle/input/datasets/sevaperetiatko/sleep-edf-final-runtime-linux
```

Подготовка offline-окружений
Оба окружения готовы; сеть для финального запуска не требуется

```
Sat Nov  7 10:50:19 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.178.04             Driver Version: 580.178.04     CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   43C    P8             10W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+
|   1  Tesla T4                       Off |   00000000:00:05.0 Off |                    0 |
| N/A   47C    P8             10W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
Notebook kernel: 3.13.15 (main, Aug  6 2026, 11:06:22) [GCC 13.3.0]
Восстановлены Python3.12.12 и3.14.2 с пакетами из Input. Скачиваний и pip/uv install нет.
```

In [4]:

```
command = [
    *BASE,
    str(FINAL_ROOT / "run.py"),
    "--data-root",
    str(DATA_ROOT),
    "--output-root",
    str(OUTPUT_ROOT),
    "--feature-python",
    str(FEATURE_PYTHON),
    "--device",
    DEVICE,
    "--threads",
    str(THREADS),
    "--feature-workers",
    str(FEATURE_WORKERS),
]
if OUTPUT_SECOND_SUBMISSION:
    command.append("--output-second-submission")
run_command(command, cwd=FINAL_ROOT, env=ENV)
report = json.loads((OUTPUT_ROOT / "run_report.json").read_text())
assert report["status"] == "EXACT_MATCH" and report["epochs"] == 66222
assert all(item["label_mismatches"] == 0 for item in report["comparisons"].values())
print("Проверено полное совпадение:", report["comparisons"])
```

Этап3/3: heads102/106 и финальные CSV 3/3 прошло 3:18

CSV созданы; проверка всех пар id,label с выбранными отправками

```
{
  "status": "EXACT_MATCH",
  "training": false,
  "raw_test_inference": true,
  "environment": {
    "python": "3.12.12 (main, Feb 12 2026, 00:42:14) [Clang 21.1.4 ]",
    "platform": "Linux-6.18.55+-x86_64-with-glibc2.39",
    "device": "cuda:0",
    "gpu": "Tesla T4",
    "torch": "2.6.0+cu124",
    "cuda": "12.4",
    "threads": 4,
    "deterministic_algorithms": true,
    "cublas_workspace_config": ":4096:8",
    "packages": {
      "numpy": "2.2.6",
      "scipy": "1.15.3",
      "scikit-learn": "1.6.1",
      "omegaconf": "2.3.0"
    }
  },
  "feature_environment": {
    "python": "3.14.2 (main, Jan 27 2026, 23:59:57) [Clang 21.1.4 ]",
    "packages": {
      "numpy": "2.5.4",
      "scipy": "1.18.1",
      "antropy": "0.2.3",
      "numba": "0.68.1",
      "llvmlite": "0.50.0",
      "scikit-learn": "1.9.1"
    }
  },
  "feature_stage_seconds": 294.46077300499996,
  "model_setup_seconds": 1.036460175000002,
  "total_seconds": 805.9195902810001,
  "epochs": 66222,
  "output_second_submission": false,
  "comparisons": {
    "157": {
      "compared_ids": 66222,
      "label_mismatches": 0,
      "examples": [],
      "output_sha256": "31dd4ec1ad8e983408b1adecb582e468a1ff9bafbbcfe4f9d48bcbe717dafc1b",
      "reference_sha256": "48962bc70c9cf75432941745eba49b48c22aadf5767c41f1e946a4f4bd34a41b",
      "semantics": "comparison only; reference predictions never replace inference outputs"
    }
  },
  "weights_manifest_sha256": "bb421a6abac6e82d7fda458ecf21166d7ffcf6cae99c75f8b33d3936a65fec59",
  "identity": {
    "implementation": "9ef7bae64bbba62d568425401e23cd4f450f46355f71d778884fb04c12c6956b",
    "weights": "bb421a6abac6e82d7fda458ecf21166d7ffcf6cae99c75f8b33d3936a65fec59",
    "data": "57982e6d757d648eedcce98ed9aa99ed6da5f494ac7269c470e7432cdb788dd6",
    "device": "cuda:0",
    "smoke_epochs": 0,
    "deterministic": true
  },
  "test_labels_read": false,
  "input_metadata": {
    "owner": "levshaazz",
    "slug": "sleep-edf-full-psg-course-data",
    "version": 1,
    "title": "Sleep-EDF Full PSG Course Data",
    "metadata_source": "https://www.kaggle.com/api/v1/datasets/view/levshaazz/sleep-edf-full-psg-course-data"
  }
}
Проверено полное совпадение: {'157': {'compared_ids': 66222, 'label_mismatches': 0, 'examples': [], 'output_sha256':
'31dd4ec1ad8e983408b1adecb582e468a1ff9bafbbcfe4f9d48bcbe717dafc1b', 'reference_sha256':
'48962bc70c9cf75432941745eba49b48c22aadf5767c41f1e946a4f4bd34a41b', 'semantics': 'comparison only; reference predictions
never replace inference outputs'}}
```


Последнюю ячейку можно выполнить отдельно после ошибки: она сохранит диагностические результаты, если run_report.json уже создан. В финальную сдачу нужен только успешный полный запуск с сохранёнными outputs. После него сохраните точную Kaggle Version, скачайте выполненный.ipynb и его PDF; добавьте version и submission IDs в метаданные.

In [5]:

```
from datetime import UTC, datetime

from IPython.display import FileLink, display
```

```
elapsed = time.perf_counter() - NOTEBOOK_STARTED
print(f"Время всех ячеек, включая установку окружений: {elapsed:.1f}s")
if (OUTPUT_ROOT / "run_report.json").exists():
    report = json.loads((OUTPUT_ROOT / "run_report.json").read_text())
    report["notebook_elapsed_seconds"] = elapsed
    report["within_two_hours_including_setup"] = elapsed <= 7200
    (OUTPUT_ROOT / "run_report.json").write_text(
        json.dumps(report, ensure_ascii=False, indent=2) + "\n"
    )
    (OUTPUT_ROOT / "input_bundle_manifest.json").write_text(
        json.dumps(manifest, ensure_ascii=False, indent=2) + "\n"
    )
    stamp = datetime.now(UTC).strftime("%Y%m%dT%H%M%SZ")
    archive = Path("/kaggle/working") / f"sleep-edf-final-157-113-results-{stamp}.zip"
    run_command(
        [
            *BASE,
            str(FINAL_ROOT / "package.py"),
            "results",
            "--results-root",
            str(OUTPUT_ROOT),
            "--output",
            str(archive),
        ],
        cwd=FINAL_ROOT,
        env=ENV,
    )
    for filename in (
        ["submission.csv", "submission_2.csv"]
        if OUTPUT_SECOND_SUBMISSION
        else ["submission.csv"]
    ):
        path = OUTPUT_ROOT / filename
        if path.exists():
            display(
                FileLink(
                    str(path.relative_to(Path.cwd())),
                    result_html_prefix=filename + ": ",
                )
            )
    display(
        FileLink(
            str(archive.relative_to(Path.cwd())),
            result_html_prefix="Все результаты одним ZIP: ",
        )
    )
    if report["status"] == "EXACT_MATCH":
        assert elapsed <= 7200, (
            "Лимит2часа превышен; run пока нельзя считать готовым финалом"
        )
```

```
Время всех ячеек, включая установку окружений: 925.9s
Results: /kaggle/working/sleep-edf-final-157-113-results-20261107T110353Z.zip;
SHA256=fd9f02c118adb6290d1b4940b14c61faa15edab479b147a6e10b67aab6d9d86a
```

submission.csv: sleep-edf-final-157-113/submission.csv
Все результаты одним ZIP: sleep-edf-final-157-113-results-20261107T110353Z.zip